

Three-Address Code Generator

This is part 2 of Exam 2 and is worth 50 points.

Your assignment is to modify the completed `cexpr` program so that it generates **three-address intermediate code** instead of evaluating C integer expressions.

Three-address code is an intermediate representation in which each instruction contains at most one operator on the right-hand side. Complex expressions are divided into a sequence of simpler instructions using temporary variables.

For example, the expression:

```
a = b + c * d;
```

should produce:

```
t1 = c * d
t2 = b + t1
a = t2
```

You will begin with a completed solution to the previous `cexpr` assignment. You should retain the scanner, grammar structure, operator precedence, associativity, and assignment syntax from that program. You will replace the semantic actions that calculate integer values with semantic actions that generate three-address code.

The program will process expressions until it encounters EOF or invalid syntax. Each expression must end with a semicolon. Tokens may be separated by whitespace. Comments are not required.

Input Language

The input language supports integer constants and the 26 lowercase variables `a` through `z`.

The operators are listed below in order of decreasing precedence.

Symbols	Description	Type	Associativity
()	Parentheses	—	—
~	Bitwise not	Unary	Right
-	Negation	Unary	Right
* / %	Multiplication, division, remainder	Binary	Left
+ -	Addition, subtraction	Binary	Left
<< >>	Shifts	Binary	Left
&	Bitwise and	Binary	Left
^	Bitwise xor	Binary	Left
	Bitwise or	Binary	Left
= += -= *= /= %= <<= >>= &= ^= =	Assignment	Binary	Right

As in the original `cexpr` assignment, no operator may precede an assignment operator in an expression except another assignment operator.

You should retain the operator precedence and associativity implemented in the provided `cexpr` solution.

The `dump;` and `clear;` commands from the original calculator are not required because this program does not store or evaluate variable values.

Three-Address Code

Each unary or binary operation must generate a new temporary variable named:

```
t1
t2
t3
...
```

Temporary-variable numbering must restart at `t1` for each input expression.

A binary operation should generate an instruction in this form:

```
temporary = operand1 operator operand2
```

For example:

```
t1 = b * c
```

A unary operation should generate an instruction in this form:

```
temporary = operator operand
```

For example:

```
t1 = -a
```

An operand may be an integer constant, a source variable, or a temporary variable.

Standalone Expressions

If an input expression is not an assignment, place its final result in the special location named `result`.

For example:

```
(a + b) * c;
```

should produce:

```
t1 = a + b
t2 = t1 * c
result = t2
```

If the expression contains only a variable or constant, no temporary variable is required.

For example:

```
a;
42;
```

should produce:

```
result = a
result = 42
```

Print a blank line after the code generated for each complete input expression.

Simple Assignment

A simple assignment should generate the code for the expression on the right-hand side and then assign its final location to the variable on the left-hand side.

For example:

```
a = b + c * d;
```

should produce:

```
t1 = c * d
t2 = b + t1
a = t2
```

If the right-hand side is a single variable or constant, no temporary is required.

```
a = b;
c = 10;
```

should produce:

```
a = b
c = 10
```

Chained Assignment

Assignment is right associative. Therefore:

```
a = b = c + 5;
```

should produce:

```
t1 = c + 5
b = t1
a = t1
```

The value produced by an assignment expression is the location assigned to the variable. This allows the result to be used by the next assignment in the chain.

Compound Assignment

Compound assignments must also generate three-address code.

For example:

```
a += b * c;
```

should produce:

```
t1 = b * c
t2 = a + t1
a = t2
```

Similarly:

```
x <<= 2;
```

should produce:

```
t1 = x << 2
x = t1
```

Compound assignments may appear in assignment chains when permitted by the original grammar.

Precedence and Associativity

The order of the generated instructions must preserve the precedence, associativity, and parentheses of the source expression.

For example:

```
a - b - c;
```

should produce:

```
t1 = a - b
t2 = t1 - c
result = t2
```

The expression:

```
a - (b - c);
```

should produce:

```
t1 = b - c
t2 = a - t1
result = t2
```

The order of the generated instructions is part of the required output.

No Evaluation or Optimization

Your program generates code but does not execute it. It should not maintain values for the variables `a` through `z`.

You are not required to detect integer overflow or division by zero because the generated operations are not evaluated.

For example:

```
a = 10 / 0;
```

should produce:

```
t1 = 10 / 0
a = t1
```

You are also not required to optimize the generated code. Do not perform constant folding, temporary reuse, algebraic simplification, or common-subexpression elimination.

For example:

```
a = 2 + 3;
```

should produce:

```
t1 = 2 + 3
a = t1
```

It should not produce `a = 5`.

Implementation

Modify the files from the `cexpr` assignment and create:

- `tac.y` (or `ex.y`)
- `scan.l` (or `ex.l`)

- Makefile

The Makefile must create an executable named:

```
tac
```

A useful approach is to change the semantic value returned by each expression nonterminal. In the original calculator, an expression returns its calculated integer value. In this assignment, an expression should return a string identifying the location where its value can be found.

That location may be:

```
42
a
t3
```

When reducing an operator production, create a new temporary variable, print an instruction, and return the temporary variable as the location of the result.

You are responsible for properly managing any dynamically allocated memory used by your implementation.

Error Handling

When the scanner encounters an invalid character or the parser encounters invalid syntax, print an appropriate syntax-error message and terminate the program.

You are not required to remove instructions that were already printed before an error was detected.

Example Input

```
a = 55 - 3;
b = c = a - 42;
a + b * c;
x += a << 2;
~(-x);
```

Example Output

```
t1 = 55 - 3
a = t1
```

```
t1 = a - 42
c = t1
b = t1
```

```
t1 = b * c
t2 = a + t1
result = t2
```

```
t1 = a << 2
t2 = x + t1
x = t2
```

```
t1 = -x
t2 = ~t1
result = t2
```

Three-address code generation complete.

Project Report

You must also submit a one-page or less single-spaced PDF report that describes:

1. In your own words, the program you set out to write.

2. How you modified the original `cexpr` solution.
3. How you debugged and tested your solution.
4. Any issues you encountered while completing the assignment.

Submission

Upload a gzipped tar file containing your source files, Makefile, and PDF report to Canvas by **11:59 p.m. on the assignment due date**.

For example:

```
tar cvzf tac.tar.gz tac.y scan.l Makefile report.pdf
```

Include any additional source or header files required to compile your program.

Partial credit will be given for incomplete efforts. However, a program that does not compile or run will receive no credit.

Point Breakdown (50 points)

- Scanner recognizes all required tokens: 5 points
- Parser accepts expressions and assignments: 8 points
- Unary and binary instructions are generated correctly: 8 points
- Simple, chained, and compound assignments work correctly: 8 points
- Precedence, associativity, and instruction order are correct: 8 points
- Output formatting and error handling: 8 points
- Project report: 5 points